



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS 216 Practical Assignment 2

- Date Issued: **3 March 2026**
 - Date Due: **23 March 2026** before **08:00**
 - Submission Procedure: **Upload to the web server (wheatley) + clickUP**
 - This assignment consists of **7 tasks** for a total of **85 marks**.
-

NB: Please read through the entire specification before asking questions on Discord/Email, as there is a high likelihood your question may be answered later in the specification.

1 Introduction

During this practical, you will be expanding your flight bookings website by loading data from an external API and implementing various new features. This will enable users of your website to browse, search and filter from a wide array of aeroplanes.

JS



2 Constraints

1. You must complete this assignment individually.
2. You may ask the Teaching Assistants for help, but they will not be able to give you the solutions.
3. You must produce all of the source files yourself; you may not use any tool to generate source files or fragments thereof automatically.
4. Your assignment will be viewed using Chrome so be sure to test your assignment in this browser. Nevertheless, you should take care to follow published standards and make sure that your assignment works in as many browsers as possible.
5. You may utilise any text editor or IDE, upon an OS of your choice, again, as long as you do not make use of any tools to generate your assignment. **(This includes ChatGPT, Gemini etc!!)**
6. All written code should contain comments including your name, surname and student number at the top of each file.
7. Your assignment must work on the **Wheatley** web server, as you will be marked off there. However, you are free and encouraged to **Develop** use a local tool like XAMPP and later upload your code to Wheatley. Make sure your website functions correctly on Wheatley, as you will use it during your demonstration.
8. **You must only use JavaScript for this assignment (no libraries with the exception of the point below).**
9. **You may use JQuery ONLY for DOM manipulation. Use of JQuery for AJAX calls will result in no marks for that section.**

3 Submission Instructions

You are required to upload all your source files (e.g. HTML5 documents, any images, etc.) to the web server (**wheatley**) and clickUP in a compressed (zip) archive. Make sure that you test your submission to the web server thoroughly. All the menu items, links, buttons, *etc.* must work and all your images must load. Make sure that your practical assignment works on the web server before the deadline. No late submissions will be accepted, so make sure you upload in good time. The server will not be accepting any uploads and updates to files from the stipulated deadline time until the end of the marking week (Thursday at 3pm).

The deadline is on Sunday but we will allow you to upload until Monday 11am. After this NO more submissions will be accepted. FTP to wheatley will also be disabled during this time.

Note, wheatley is currently available from anywhere. But do not rely that outside access from the UP network will always work as intended. You must therefore make sure that you **ftp** your assignment to the web server. Also make sure that you do this in good time. A snapshot of the web server will be taken just after the submission was due and only files in the snapshot will be marked.

Practicals are marked by demonstrating your practical to a tutor during the allotted marking weeks. **If you do not demonstrate your practical you will receive 0 for the practical.** You will only be marked in the practical session that you have booked on the cs portal, if you miss it, you will receive 0 (Unless special permissions have been granted by the lecturer. i.e. You were sick and able to provide a sick note).

4 Online resources

HTML5 - <https://developer.mozilla.org/en-US/docs/Web/Guide/HTML/HTML5>
<http://www.w3schools.com/>

CSS - <http://www.w3.org/Style/Examples/011/firstcss>

Javascript - <https://www.w3schools.com/js/>
https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/JavaScript_basics

REST - https://en.wikipedia.org/wiki/Representational_state_transfer

JSON - https://www.w3schools.com/js/js_json_syntax.asp

HTTP Methods - https://www.w3schools.com/tags/ref_httpmethods.asp

AJAX - https://www.w3schools.com/js/js_ajax_intro.asp

5 Rubric for marking

API Functionality	20
AJAX	15
JSON manipulation	5
Book Flights Page	20
Data population	10
DOM manipulation	10
Planes Page	20
Data population	5
Searchable	5
Filterable	5
Sort-able	2
Loading Screen	3
DOM manipulation	5
View page	10
Correct plane details displayed	2
Additional Plane Info	3
DOM Manipulation	5
Favourite Planes page	10
DOM Manipulation	8
Choice for local/session storage	2
Validation	5
HTML	3
JS	2
Upload	
Does not work on wheatley	-20
Not uploaded to clickUP	-85
Not Demoed	-85
Bonus	10
Total	85

6 Assignment Instructions

Task 1: Populate from APIs (40 marks)

This task requires you to pull information from a **Wheatley API** and populate all the templates (HTML layouts) you have created in PA 1 for the Book Flights and Planes pages. You should retrieve data from the Wheatley API for at least 20 planes on the "Planes" page.

All API calls must be done through JavaScript **AJAX XMLHttpRequest** (https://www.w3schools.com/js/js_ajax_intro.asp). You must choose between synchronous and asynchronous calls and provide reasons for your choice.

NB: Your reasoning should be included at the top of your JS file/s as a comment.

For this practical you will use an API Hosted off Wheatley. You must use this API and no others (except the flagsAPI). The documentation and usage of said API can be found at <https://wheatley.cs.up.ac.za/api/doc.html>.

You will have to navigate to: <https://wheatley.cs.up.ac.za/api/generate.html>, to create your API key. You will have to use this key and your student number to use the API!

The following functionality needs to be implemented:

- You need to **replace your mock data** from PA 1 with data from the API that will load dynamically when the user views the page.
- For all dynamic content you need to have a **loading screen/animation** to show that the data is being retrieved/processed. This can be a GIF/APNG/SVG loader for each element or a loading screen for the entire page (it will disappear once all the data on that page has loaded), Decide which is best. You can get resources from (<https://loading.io/>) however you are not limited to that resource.

Task 2: Update the "Book Flight" page (20 marks)

Since there are thousands of airports worldwide, loading them all into a dropdown at once would crash the browser or cause massive lag. To overcome this do the following:

- Lazy Loading: Instead of fetching 5,000 airports, the dropdown initially loads only a small batch (e.g., the first 20). As the user scrolls down, the next 20 are fetched and appended.
- Multi-Criteria Search: The search bar shouldn't just look for "London." It needs to filter/search based on four distinct attributes:
 - Name: "Heathrow"
 - City: "London"
 - Country: "United Kingdom"
 - Airport Code: "LHR"

The "Cabin Class" selection is now dependent on the "Plane" selection. You cannot show classes that the specific aircraft does not support.

The Selection Flow

1. Retrieve Plane: Use a search bar to select a specific PlaneID.
2. Filter Classes: Once a plane is selected, query the Cabins table where the link matches that aircraft.
3. Update UI: Map the results to the Cabin dropdown.

Task 3: Update the "Planes" page (20 marks)

This page should now retrieve at least 20 planes from the API The following should also be completed on the Planes page:

- Search from the previous practical must be implemented using the API. Use the search bar to search for a specific plane based on its manufacturer.
- Sort from the previous practical must be implemented using the API.
- Filter from the previous practical must be implemented using the API.
- You should also implement a way to indicate to the user if no results were found from a search and/or filter.

Task 4: Update the "View" page (10 marks)

The view page from the previous practical should be implemented. Have a look at the PA 1 specification to remind you what should be present on this page. Since the page should be implemented, remove mock data (if present) and check if the page works correctly. It is important that even when the page refreshes, the information displayed does not change. **Hint:** Remember query parameters

Task 5: Update the "Favourites" page (10 marks)

The Favourites page from the previous practical should now be implemented. You should store the id's of favourited planes in local storage/session storage, you should be able to explain why you made the choice of which ever one you decided to use. **Hint:** Have a look at LocalStorage and SessionStorage in JS.

Task 6: Validation (5 marks)

All HTML and JS should be valid and comply with standards.

Task 7: Bonus (10 marks)

Bonus marks will be given for additional functionality and awarded based on the difficulty level. These marks will only be awarded for notable additions (simply adding a background image is not enough). Bonus marks are capped to full marks.

The following, among others, may earn you extra marks:

- Calculate the number of planes with a certain number of seats
- Add an "About us" page, with Frequently Asked Questions